

→ SELECT * FROM table ORDER BY department, salary DESC

First
in Ascending order

second
descending

Double ""

→ operator precedence

L → R

* , / → higher precedence

+ , - → lower precedence

SELECT 10 + 5 * 2 AS "result" FROM dual
$$\frac{10 + 10}{20}$$

if both present then Left to right

⇒ 100 / 5 * 2
20 * 2
40

→ LIKE, NOT LIKE

where name LIKE '%S_C'

escape character '\' → '_' → escaping 'DBS_%' → DBS_XYZZY

→ where Price BETWEEN 1000 AND 2000;

— NOT BETWEEN

→ where city IN ('HR', 'CAO', 'US')

NOT IN

→ IS NULL, IS NOT NULL

→ where category_id = 4 AND list_price < 200;
category_id = 3 OR category_id IN (5, 6, 7);

→ PRECEDENCE

Paranthesis → * / → + - → =, <, >, <=, >=, !=, <> → IS NULL, IS NOT NULL, LIKE, IN

OR < AND < NOT < BETWEEN

→ first_name || ' ' → ' ' || last_name AS "Full Name" CONCAT(CONCAT(first_name, ' '), last_name)

→ last_name || ' ' || job_title → Smith's job is Manager

→ DESC tableName
DESCRIBE tableName.
colName, NULL, TYPE
P_id NOT NULL NUMBER

→ LOWER, UPPER, INITCAP → Parth Patel Ranjithan

CONCAT, SUBSTR ('Database Systems', 1, 4)
('a', 'b')
1 2 3 4
Pos How many chars from that pos

LENGTH ('PARTH')
↳ 5

SUBSTR ('Database Systems', -4, 2)
-4 -3 -2 -1
1 2
('te')

INSTR ('Database Systems', 'b')
1 2 3 4 5
↳ And position of first occurrence of substring

LPAD ('Tommy', 10, '*')
total length

RPAD ('Tommy', 10, '*')
Tommy*****

*****Tommy
1 2 3 4 5 6 7 8 9 10
left

REPLACE ('Jack and Jue', 'J', 'B')
B1
↳ Black and Blue

TRIM ('a' FROM 'aabbccag')

(15.19) → 15
(15.19, 2) → 15.19
bbcc

TRUNC (15.193, 2) → 15.19
(15.193, 1) → 15.1

(12345.678, -2) → 12300

(12345.678, -3) → 12000

(12888.678, -3) → 12000

↳ no rounding

MOD (3, 2) → $\frac{3}{2}$
1 → mod

NVL (state, 'unknown')

↓
if state is null
then replace with 'unknown'

NVL2 (ex, value if not NULL, value if NULL)

(state, 'state is known', 'state is unknown')

COALESCE (state, city, 'unknown')

↓ ↓
if state is null then return city
if state and city both null then return 'unknown'

select SYSDATE from dual
select * from v\$NLS_PARAMETERS; - return all National Language Support

current_date + 1 → next day

select DATE '2018-05-13' from dual - return this date

select empid, (sysdate - hiredate) / 7 as "weeks"

→ add_months(hiredate, 1)

select current_timestamp from dual

select extract(year from current_date) from dual
day
month

select last_day(current_date) from dual → 2025-02-28
(show last day of current month)

select trunc(current_date - hire_date) from table { select trunc(months_between(current_date, hiredate), 0)
days between
return fractional
months between
rounded to 1 day of month

select round(sysdate, 'MONTH') from dual → today is 2025-02-17 then round 2025-03-01

'YEAR'

rounded to
→ first day of year 2025-01-01

trunc(sysdate, 'MONTH') from dual → 01-02-25

'YEAR'

→ 01-02-25

select next_day(current_date, 'Friday') → find when next Friday from current_date

TO_CHAR(sysdate, 'YYYY-DD-MM') 2025-05-11

'YYYY-DD-MON') 16-07-JAN

'MONTH'

JUNE

'DAY'

Tuesday

'DD'

13

'YYYY-Month-DD'

2016-June-07

month

June

put extra
space
remove
zero

'fmYYYY-Month-DD'

2016-June-7

'MM/YY'

06/25

'fm MM/DD/YY'

--- 6/25/16



'WW') || 'th week of the year'

'W') || 'th week of month'

'DD "of" MONTH' → 17 of FEBRUARY

'DD^{spth} "of" MONTH' - EIGHTEENTH of FEBRUARY

remove space → 'FM DD-MONTH-YYYY'

'Dd^{spth} "of" Month HH:MI' → Eighteenth of February 12:52

9 87643257.99 , '\$ 999,999,999.99'

↳ \$ 987,673,257.99
↳ \$ 73,257.99

TO-NUMBER ('1234')

TO-DATE ('09-01-2020', 'DD-MM-YYYY')

('09-SEP-16', 'DD-MON-YY')

'May 24, 2016', 'Month DD, YYYY')

select hiredate, round (months_between (current_date, hiredate)) from table where hire date

in November { BETWEEN TO-DATE ('01-11-2016', 'DD-MM-YYYY')
month, 2016 } AND TO-DATE ('30-11-2016--')

• INSERT:

- The INSERT statement adds new records to a table.
- Syntax: INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);
- Example: INSERT INTO employees (first_name, last_name, salary) VALUES ('John', 'Doe', 50000);

• UPDATE:

- The UPDATE statement modifies existing records in a table.
- Syntax: UPDATE table_name SET column1 = value1, column2 = value2, ... WHERE condition;
- Example: UPDATE employees SET salary = 55000 WHERE first_name = 'John';

• DELETE:

- The DELETE statement removes records from a table.
- Syntax: DELETE FROM table_name WHERE condition;
- Example: DELETE FROM employees WHERE last_name = 'Doe';

```
ADD - new column
ALTER TABLE customer
ADD age INT NOT NULL;

MODIFY - Change datatype of an attribute.
ALTER TABLE customer
MODIFY price DOUBLE;
-- If you want to keep the existing data type, you can omit the data type specification, and MySQL will retain the existing data type.

CHANGE column name
ALTER TABLE customer
CHANGE COLUMN new_customer_name VARCHAR(100);
-- If you want to keep the existing data type, you can omit the data type specification, and MySQL will retain the existing data type.

DROP COLUMN
ALTER TABLE customer
DROP COLUMN col_name;

RENAME the table
ALTER TABLE oldTableName RENAME TO newTableName;
```

```
CREATE OR REPLACE PROCEDURE add_numbers_proc (
    a IN NUMBER,
    b IN NUMBER,
    result OUT NUMBER
)
AS
    dummy_result NUMBER; -- • Dummy (local) variable inside AS
BEGIN
    dummy_result := a + b; -- Use dummy variable
    result := dummy_result; -- Assign to OUT param
    DBMS_OUTPUT.PUT_LINE('Inside Procedure: a + b = ' || dummy_result);
END;
```

fn AS

```
CREATE OR REPLACE FUNCTION add_numbers_func (
    a IN NUMBER,
    b IN NUMBER
)
RETURN NUMBER
AS
    res NUMBER;
BEGIN
    res := a + b;
    DBMS_OUTPUT.PUT_LINE('Inside Function: a + b = ' || res);
    RETURN res;
END;
```

How to Call It:

```
sql
DECLARE
    num1 NUMBER := 10;
    num2 NUMBER := 20;
    res NUMBER;
BEGIN
    add_numbers_proc(num1, num2, res);
    DBMS_OUTPUT.PUT_LINE('Outside Procedure: Result is ' || res);
END;
```

fn i

Step 2: Call the function and display result outside too

```
sql
DECLARE
    num1 NUMBER := 15;
    num2 NUMBER := 25;
    result NUMBER;
BEGIN
    result := add_numbers_func(num1, num2);
    DBMS_OUTPUT.PUT_LINE('Outside Function: Result is ' || result);
END;
```

```
CREATE OR REPLACE PROCEDURE remove_employee AS
    managerId NUMBER;
BEGIN
    managerId := 2;

    DELETE FROM new_employee
    WHERE manager_id = managerId;

    IF SQL%ROWCOUNT = 0 THEN
        DBMS_OUTPUT.PUT_LINE('No employee is deleted');
    ELSIF SQL%ROWCOUNT = 1 THEN
        DBMS_OUTPUT.PUT_LINE('One employee is deleted.');
```

SQLERRM

catch all exceptions

```
DECLARE
    semester CHAR(1);
BEGIN
    semester := 'S';

    CASE semester
        WHEN 'F' THEN DBMS_OUTPUT.PUT_LINE('Fall Term');
        WHEN 'M' THEN DBMS_OUTPUT.PUT_LINE('Winter Term');
        WHEN 'S' THEN DBMS_OUTPUT.PUT_LINE('Summer Term');
        ELSE DBMS_OUTPUT.PUT_LINE('Wrong Value');
    END CASE;
END;
```

Exp. WHEN CASE-NOT-FOUND

```
DECLARE
    productId NUMBER := 2; -- Input: Product ID to search
    productName VARCHAR2(255); -- To store the product name
    price NUMBER(9,2); -- To store the product price
BEGIN
    -- Try to fetch product name and price from products table
    SELECT product_name, list_price
    INTO productName, price
    FROM products
    WHERE product_id = productId;

    -- Output the result if found
    DBMS_OUTPUT.PUT_LINE('Product Name: ' || productName);
    DBMS_OUTPUT.PUT_LINE('Product Price: ' || price);

    -- Exception handling block starts
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('No product found with ID ' || productId);
        WHEN TOO_MANY_ROWS THEN
            DBMS_OUTPUT.PUT_LINE('More than one product found for ID ' || productId);
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
END;
```

DATA TYPE inside PL
NUMBER, VARCHAR2, BOOLEAN, DATE
varName tableName.colName %TYPE

```
DECLARE
    counter NUMBER := 3;
BEGIN
    DBMS_OUTPUT.PUT_LINE('---- Count Down ----');
    LOOP
        DBMS_OUTPUT.PUT_LINE('counter: ' || counter);
        counter := counter - 1;
        IF counter < 1 THEN
            EXIT;
```

```
...
LOOP
    DBMS_OUTPUT.PUT_LINE('counter: ' || counter);
    counter := counter - 1;
    EXIT WHEN counter < 1;
END LOOP;
```

```
DECLARE
    counter NUMBER := 4;
BEGIN
    DBMS_OUTPUT.PUT_LINE('---- Count Down ----');
    LOOP
        counter := counter - 1;
        IF counter = 2 THEN
            CONTINUE;
```

CONTINUE WHEN counter = 2;

per only

```
FOR i IN [REVERSE] lower..upper LOOP
    -- statements
END LOOP;
```

Example:

```
sql
BEGIN
    FOR i IN 1..4 LOOP
        IF i < 2 THEN
            DBMS_OUTPUT.PUT_LINE(i || ' is less than 2');
        ELSIF i > 2 THEN
            DBMS_OUTPUT.PUT_LINE(i || ' is greater than 2');
        ELSE
            DBMS_OUTPUT.PUT_LINE(i || ' is equal to 2');
        END IF;
    END LOOP;
END;
```

```
WHILE condition LOOP
    -- statements
END LOOP;
```

Example:

```
sql
DECLARE
    run BOOLEAN := TRUE;
    round NUMBER := 1;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- First WHILE LOOP ---');
    WHILE run LOOP
        DBMS_OUTPUT.PUT_LINE('round ' || round);
        round := round + 1;
        IF round = 4 THEN
            run := FALSE;
        END IF;
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('--- Second WHILE LOOP ---');
    WHILE NOT run LOOP
        DBMS_OUTPUT.PUT_LINE('round ' || round);
        round := round - 1;
        IF round = 0 THEN
            run := TRUE;
        END IF;
    END LOOP;
END;
```

1. Implicit Cursor

Used with:

- INSERT, UPDATE, DELETE, and SELECT INTO

Common Attributes:

Attribute	Description
<code>SQL%FOUND</code>	TRUE if at least one row was affected
<code>SQL%NOTFOUND</code>	TRUE if no rows were affected
<code>SQL%ROWCOUNT</code>	Number of rows affected
<code>SQL%ISOPEN</code>	Always FALSE for implicit cursors

Example:

```
plsql
DECLARE
  emp_name employees.last_name%TYPE;
BEGIN
  SELECT last_name INTO emp_name
  FROM employees
  WHERE employee_id = 100;

  IF SQL%FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Employee: ' || emp_name);
  END IF;
END;
```

Example: Basic Explicit Cursor

```
plsql
DECLARE
  e_last_name employees.last_name%TYPE;
  e_job_title employees.job_title%TYPE;

  CURSOR emp_cursor IS
    SELECT last_name, job_title
    FROM employees
    WHERE job_title LIKE 'AN%'
    ORDER BY last_name;
BEGIN
  OPEN emp_cursor;
  LOOP
    FETCH emp_cursor INTO e_last_name, e_job_title;
    EXIT WHEN emp_cursor%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE('Employee: ' || e_last_name || ' ' || e_job_title);
  END LOOP;
  CLOSE emp_cursor;
END;
```

3. Explicit Cursor with Parameters

Cursors can take input parameters just like procedures.

Example:

```
plsql
DECLARE
  p_product products.product%TYPE;
  p_price products.price%TYPE;

  CURSOR product_cursor (p_product_name products.product%TYPE) IS
    SELECT * FROM products
    WHERE list_price BETWEEN p_price AND 100;
BEGIN
  OPEN product_cursor('100', 100);
  LOOP
    FETCH product_cursor INTO p_product;
    EXIT WHEN product_cursor%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE('Product: ' || p_product || ' Price: ' || p_price);
  END LOOP;
  CLOSE product_cursor;
END;
```

4. Explicit Cursor with FOR LOOP

No need to manually open/fetch/close the cursor — PL/SQL does it for you.

Example:

```
plsql
DECLARE
  CURSOR emp_cursor IS
    SELECT last_name, job_title FROM employees
    WHERE job_title LIKE 'AN%'
    ORDER BY last_name;
BEGIN
  FOR item IN emp_cursor LOOP
    DBMS_OUTPUT.PUT_LINE('Employee: ' || item.last_name || ', Job: ' || item.job_title);
  END LOOP;
END;
```

5. Cursor Attributes

Attribute	Description
<code>%ISOPEN</code>	TRUE if the cursor is open
<code>%FOUND</code>	TRUE if last fetch returned a row
<code>%NOTFOUND</code>	TRUE if last fetch did NOT return a row
<code>%ROWCOUNT</code>	Number of rows fetched

Create a Database

```
js
use myNewDB
db.newCollection.insertOne({ field: "value" })
```

Drop a Database

```
js
db.dropDatabase()
```

Create and Remove Documents

Insert One Document

```
js
db.blog.insertOne({
  title: "My Blog Post",
  content: "Here's my blog post.",
  date: new Date()
})
```

Insert Many Documents

```
js
db.blog.insertMany([
  { title: "Post 1", content: "Content 1" },
  { title: "Post 2", content: "Content 2" }
])
```

Bulk Insert with Validation

- MongoDB prevents duplicate `_id` entries.
- Use `continueOnError` to skip failed inserts and continue.

Remove Documents

```
js
db.foo.remove() // Remove all
db.users.remove({ active: false }) // Conditional remove
```

Step 1: Create and View the Database

```
bash
> use dummyDB
switched to db dummyDB
```

Step 2: Insert Documents into users Collection

```
bash
> db.users.insertMany([
  { _id: 1, username: "john", age: 27, email: "john@example.com", registered: new Date("2023-01-01") },
  { _id: 2, username: "mary", age: 22, email: "mary@example.com", registered: new Date("2023-01-01") },
  { _id: 3, username: "alice", age: 30, email: "alice@example.com", registered: new Date("2023-01-01") },
  { _id: 4, username: "bob", age: 18, email: "bob@example.com", registered: new Date("2023-01-01") }
])
```

```
bash
{
  acknowledged: true,
  insertedIds: { '0': 1, '1': 2, '2': 3, '3': 4 }
}
```

Find All Users

```
bash
> db.users.find()

Result:
{ "_id": 1, "username": "john", "age": 27, "email": "john@example.com", "registered": ISODate("2023-01-01T00:00:00Z") }
{ "_id": 2, "username": "mary", "age": 22, "email": "mary@example.com", "registered": ISODate("2023-01-01T00:00:00Z") }
{ "_id": 3, "username": "alice", "age": 30, "email": "alice@example.com", "registered": ISODate("2023-01-01T00:00:00Z") }
{ "_id": 4, "username": "bob", "age": 18, "email": "bob@example.com", "registered": ISODate("2023-01-01T00:00:00Z") }
```

Match by Key/Value

```
bash
db.users.find({ age: 27 })
```

- Finds users where `age` is exactly 27.

```
bash
db.users.find({ username: "joe", age: 27 })
```

- Finds users with both `username = "joe"` and `age = 27`.

Project Specific Fields

```
bash
db.users.find({}, { username: 1, email: 1, _id: 0 })
```

- Show only `username` and `email`, hide `_id`.

```
bash
db.users.find({}, { in_stock: 0 })
```

- Exclude `in_stock` from the results.

Comparison Operators

```
bash
db.users.find({ age: { $gt: 20 } })
```

- Finds users where `age` is greater than 20.

```
bash
db.users.find({ age: { $gte: 18, $lte: 30 } })
```

- Finds users with `age` between 18 and 30 inclusive.

```
bash
db.users.find({ username: { $ne: "joe" } })
```

- Finds users whose username is not 'joe'.

\$in and \$nin Operators

```
bash
db.users.find({ age: { $in: [19, 27] } })
```

- Finds users where `age` is either 19 or 27.

```
bash
db.users.find({ age: { $nin: [19, 27] } })
```

- Finds users where `age` is not 19 or 27.

\$or and \$and Operators

```
bash
db.users.find({ $or: [ { age: 19 }, { in_stock: 0 } ] })
```

- Finds users where `age` is 19 or `in_stock` is 0.

```
bash
db.users.find({ $and: [ { in_stock: { $gt: 0 } }, { age: { $gte: 27 } } ] })
```

- Finds users where `in_stock` is greater than 0 and `age` is greater than or equal to 27.

\$not Operator

```
bash
db.users.find({ $not: { $gt: 30 } })
```

- Finds users where `age` is not greater than 30.

\$null and \$exists Operators

```
bash
db.users.find({ age: null })
```

- Finds documents where `age` is null or missing.

```
bash
db.users.find({ $in: [null], $exists: true })
```

- Finds documents where `age` exists and is null.

```
bash
db.users.find({ email: { $exists: false } })
```

- Finds documents missing the `email` field.

Array Queries

```
bash
db.foo.find({ fruits: { $all: ["apple", "banana"] } })
```

- Finds docs where `fruits` array contains both 'apple' and 'banana'.

```
bash
db.foo.find({ fruits: { $size: 3 } })
```

- Finds docs where `fruits` array has exactly 3 elements.

Embedded Document Queries

```
bash
db.people.find({ name: { first: "John", last: "Snow" } })
```

- Match entire embedded document.

```
bash
db.people.find({ "name.first": "John" })
```

- Match individual nested fields.

\$elemMatch for Embedded Arrays

Example Document in blog Collection

```
js
{
  _id: 1,
  title: "Hello World",
  comments: [
    { author: "John", score: 3, comment: "Nice post!" },
    { author: "Mary", score: 6, comment: "Terrible post!" }
  ]
}
```

WRONG way (won't work)

```
bash
db.blog.find({ comments: { author: "John", score: { $gte: 5 } } })
```

- Will not match even if both fields exist in array — it doesn't check them together.

CORRECT way using \$elemMatch

```
bash
db.blog.find({
  comments: {
    $elemMatch: {
      author: "John",
      score: { $gte: 5 }
    }
  }
})
```

- Matches:

`{_id: 1}`, because one comment object matches both conditions

1. Document Replacement

Example:

```
js
{ name: "joe" }
```

Replace whole document:

```
bash
db.people.replaceOne(
  { name: "joe" },
  { name: "joe", friends: 32, enemies: 2 }
)
```

✓ Replaces the entire document. Any fields not mentioned (e.g., `_id`, `age`) are removed unless included again.

2. Update Document Schema (Restructure)

Example:

```
js
{
  _id: ObjectId("..."),
  name: "joe",
  friends: 32,
  enemies: 2
}
```

Restructure using code:

```
js
joe = db.users.findOne( { name: "joe" } )
joe.relationships = { friends: joe.friends, enemies: joe.enemies }
joe.username = joe.name
delete joe.friends
delete joe.enemies
delete joe.name
db.users.update( { _id: joe._id }, joe )
```

✓ Converts flat fields into a nested `relationships` object and renames `name` to `username`.

3. Duplicate Key Error

If you do this:

```
bash
db.people.update( { name: "joe" }, joe )
```

It will fail if more than one `joe` exists — MongoDB tries to apply the `_id` from one document to another, causing a duplicate key error.

✓ Always match documents using `_id` for updates.

4. Update Modifiers

\$inc – Increment numbers

Example:

```
{ url: "www.example.com", pageviews: 52 }
```

Increment pageviews:

```
bash
db.analytics.update(
  { url: "www.example.com" },
  { $inc: { pageviews: 1 } }
)
```

✓ Updates `pageviews` to 53.

\$set – Add or update fields

Add a new field:

```
bash
db.users.update(
  { _id: ObjectId("..." ) },
  { $set: { "favorite book": "War and Peace" } }
)
```

Modify existing field:

```
bash
db.users.update(
  { name: "joe" },
  { $set: { "favorite book": "Seven Eggs and Ham" } }
)
```

Set an array:

```
bash
db.users.update(
  { name: "joe" },
  { $set: { "favorite book": [ "Cat's Cradle", "Foundation Trilogy", "Ender's Game" ] } }
)
```

\$unset – Remove a field

```
bash
db.users.update(
  { name: "joe" },
  { $unset: { "favorite book": 1 } }
)
```

✓ Deletes the `favorite book` field from the document.

\$set – On embedded document

Before:

```
js
author: { name: "joe", email: "joe@example.com" }
```

Update nested field:

```
bash
db.blog.posts.update(
  { "author.name": "joe" },
  { $set: { "author.name": "Joe Schmoe" } }
)
```

✓ Changes only the `author.name` field.

5. Array Modifiers

\$push – Add item to array

Add new comment:

```
bash
db.blog.posts.update(
  { title: "A blog post" },
  { $push: { comments: { name: "joe", email: "joe@example.com", content: "nice post." } } }
)
```

✓ Adds the comment. If `comments` array doesn't exist, MongoDB creates it.

\$push – Add another comment:

```
bash
db.blog.posts.update(
  { title: "A blog post" },
  { $push: { comments: { name: "bob", email: "bob@example.com", content: "good post." } } }
)
```

✓ Appends to existing `comments` array.

\$each – Push multiple items at once

```
bash
db.stock.ticker.update(
  { _id: "GOOG" },
  { $push: { hourly: { $each: [562.776, 562.798, 559.121] } } }
)
```

✓ Adds 3 numbers to the `hourly` array.

\$slice – Limit array length

```
bash
db.movies.update(
  { genre: "horror" },
  { $push: { top10: { $each: [ "Saw", "Nightmare" ], $slice: -10 } } }
)
```

✓ Only last 10 items are kept in `top10`.

\$sort – Sort array while pushing

```
bash
db.movies.update(
  { genre: "horror" },
  {
    $push: {
      top10: {
        $each: [
          { name: "Nightmare", rating: 6.6 },
          { name: "Saw", rating: 4.3 }
        ],
        $sort: { rating: -1 },
        $slice: 10
      }
    }
  }
)
```

✓ Sorts by `rating` descending and keeps top 10.

6. Arrays as Sets (Avoid duplicates)

Manual way using `$ne`:

```
bash
db.papers.update(
  { "authors cited": { $ne: "Richie" } },
  { $push: { "authors cited": "Richie" } }
)
```

✓ Only pushes `"Richie"` if not already in the array.

Cleaner way with `$addToSet`:

Starting array:

```
js
emails: [ "joe@example.com", "joe@gmail.com", "joe@yahoo.com" ]
```

Try to add duplicate:

```
bash
db.users.update(
  { _id: ObjectId("..." ) },
  { $addToSet: { email: "joe@gmail.com" } }
)
```

✗ No change (already exists)

Add a new unique value:

```
bash
db.users.update(
  { _id: ObjectId("..." ) },
  { $addToSet: { email: "joe@hotmail.com" } }
)
```

✓ `"joe@hotmail.com"` is added.